CSC 2105: Data Structure

Pointer

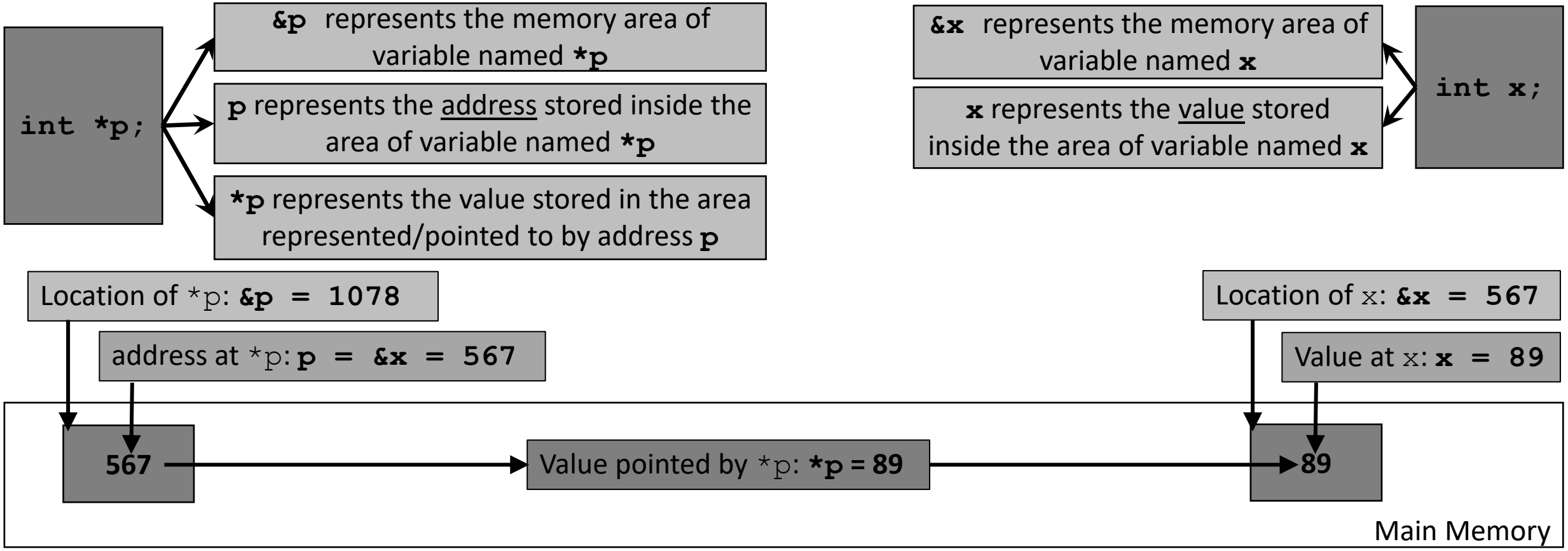
Fahad Ahmed
Assistant Professor, Department of Computer Science
Faculty of Science & Information Technology
American International University-Bangladesh (AIUB)
fahad.ahmed@aiub.edu

VARIABLE & POINTER

- The computer access its own memory not by using variable names but by using a memory map where each location of memory is uniquely defined by a number, called the *address*. Pointers are a very powerful, but primitive facility to avail that address. To understand pointer let us go through the concept of variables once more.
- A *variable* is an area of *memory* that has been given a name. For example: `int x;` is an area of memory that has been given the name `x`. The instruction `x=10;` stores the data value `10` in the area of memory named `x`. The instruction `&x` returns the *address* of the location of variable `x`.
- A *pointer* is a variable that *stores the location of a memory/variable*. A pointer has to be declared. For example: `int *p;` Adding an asterisk (called the *de-referencing* operator) in front of a variable's name declares it to be a pointer to the declared type.
- `int *p;` is a pointer – which can store an address of a memory location where an integer value can be stored or which can store an address of the memory location of an integer variable.
- For example: `int *p , q;` declares `p`, a pointer to `int`, and `q` an `int` and the instruction: `p=&q;` stores the address of `q` in `p`. After this instruction, conceptually, `p` is *pointing* at `q`.

VARIABLE & POINTER…

- After declaring a pointer `*p` variable, it can be used like any other variable. That is, `p` stores the *address* or *pointer*, to another variable; `&p` gives the address of the pointer variable itself; and `*p` is the *value* stored in the variable that `p` *points* at.



```cpp
// Understanding pointer variable
void main( void )
{
 int x = 10;
 int* p = &x;
 int y = *p;
 cout <<"Address of integer variable x: "<< &x <<"\n";
 cout <<"Value stored in the memory area of x: "<< x <<"\n";
 cout <<"Address of integer pointer variable *p: "<< &p <<"\n";
 cout <<"Address stored in the area of pointer *p: "<< p<<"\n";
 cout <<"Address of integer variable y: "<< &y <<"\n";
 cout <<"Value pointed to by the pointer *p: "<< *p <<"\n";
 cout <<"Value stored in the memory area of variable y: "<< y <<"\n";
}
```

```
Address of integer variable x: 0x8fbbfff0
Value stored in the memory area of x: 10
Address of integer pointer variable *p: 0x8fbbfff4
Address stored in the area of pointer *p: 0x8fbbfff0
Address of integer variable y: 0x8fbbfff8
Value pointed to by the pointer *p: 10
Value stored in the memory area of variable y: 10
```

variable	Memory Address	value	
int x	0x8f86fff0	10	x=10 &x=0x8f86fff0
	0x8f86fff1		
	0x8f86fff2		
	0x8f86fff3		
int *p	0x8f86fff4	0x8f86fff0	p=0x8f86fff0 &p=0x8f86fff4 *p=*(0x8f86fff0) =*(&x)=x=10
	0x8f86fff5		
	0x8f86fff6		
	0x8f86fff7		
int y	0x8f86fff8	10	y=*p=10 &y=0x8f86fff8
	0x8f86fff9		
	0x8f86fffa		
	0x8f86fffb		

POINTER & ARRAY

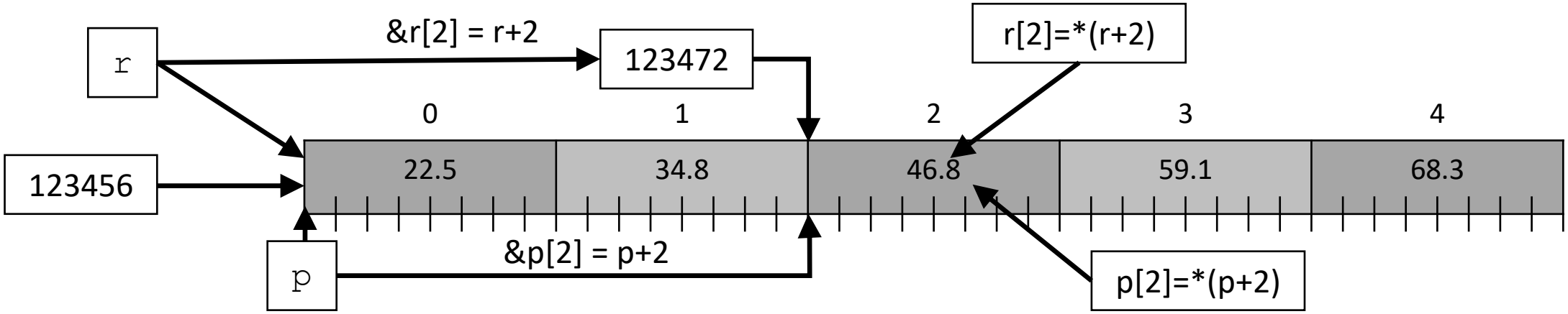
```
1  void main( void )
2  {
3   float r[5] = {22.5,34.8,46.8,59.1,68.3};
4   cout <<"1st element: "<< r[0] <<"\n";
5   cout <<"1st element: "<< *r <<"\n";
6   cout <<"3rd element: "<< r[2] <<"\n";
7   cout <<"3rd element: "<< *(r+2)<<"\n";
8   float *p;
9   p = r; //&r[0]
10  cout <<"1st element: "<< p[0] <<"\n";
11  cout <<"1st element: "<< *p <<"\n";
12  cout <<"3rd element: "<< p[2]<<"\n";
13  cout <<"3rd element: "<< *(p+2)<<"\n";
14  for(int i=0, p=r; i<5; i++, p++)
15    cout <<"Element "<<(i+1)<<" is: "<<*p<<"\n";
16 }
```

An array is simply a block of memory. An array can be accessed with pointers as well as with [] square brackets. *The name of an array variable is a pointer to the first element in the array.* So, any operation that can be achieved by array subscripting can also be done with pointers or vice-versa.

```
1st element: 22.5
1st element: 22.5
3rd element: 46.8
3rd element: 46.8
1st element: 22.5
1st element: 22.5
3rd element: 46.8
3rd element: 46.8
Element 1 is: 22.5
Element 2 is: 34.8
Element 3 is: 46.8
Element 4 is: 59.1
Element 5 is: 68.3
```

POINTER & ARRAY

- The array `float r[5];` or the pointer variable `*p` (after `p=r`) is a pointer to the first floating point number in the declared array.
- The 1st element of the array, `22.3`, can be accessed by using: `r[0]`, `p[0]`, `*r` or `*p`.
- The 3rd element, `46.8`, could be accessed by using: `r[2]`, `p[2]`, `*(r+2)` or `*(p+2)`.
- Now, let's examine the notation `(r+2)` and `(p+2)`.



- Assuming the starting address of the array numbers is 123456 –

`r[0]=(r+0)` starts at address, `r+0*sizeof(float)` = 123456 + 0 * 8 = 123456,
`r[1]=(r+1)` starts at address, `r+1*sizeof(float)` = 123456 + 1 * 8 = 123464,
`r[2]=(r+2)` starts at address, `r+2*sizeof(float)` = 123456 + 2 * 8 = 123472.

void POINTERS

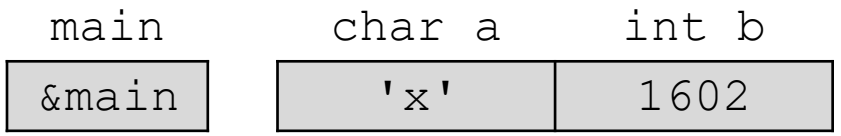
```
1  // increaser                                          Y, 1603
2  void increase(void *data, int psize){
3    if ( psize == sizeof(char) ){
4       char *pchar;
5       pchar=(char*)data;
6       ++(*pchar);
7    }
8    else if (psize == sizeof(int)){
9       int *pint;
10      pint=(int*)data;
11      ++(*pint);
12   }
13 }
14 void main (void){
15   char a = 'x';
16   int b = 1602;
17   increase (&a,sizeof(a));
18   increase (&b,sizeof(b));
19   cout << a << ", " << b << endl;
20 }
```

- The `void` type of pointer is a special type of pointer which represents the absence of type. So `void` pointers are pointers that point to a value that has no type (and thus also an undetermined length and undetermined dereference properties).
- This allows `void` pointers to point to any data type, `int`, `float`, `char`, `double` or any type of array.
- But the data pointed by them cannot be directly dereferenced, since we have no type to dereference to.
- So need to *cast* the address in the `void` pointer to some other pointer type that points to a concrete data type before dereferencing it.

void POINTERS

```
1  // increaser
2  void increase(void *data, int psize){
3    if ( psize == sizeof(char) ){
4       char *pchar;
5       pchar=(char*)data;
6       ++(*pchar);
7    }
8    else if (psize == sizeof(int)){
9       int *pint;
10      pint=(int*)data;
11      ++(*pint);
12   }
13 }
14 void main (void){
15   char a = 'x';
16   int b = 1602;
17   increase (&a,sizeof(a));
18   increase (&b,sizeof(b));
19   cout << a << ", " << b << endl;
20 }
```

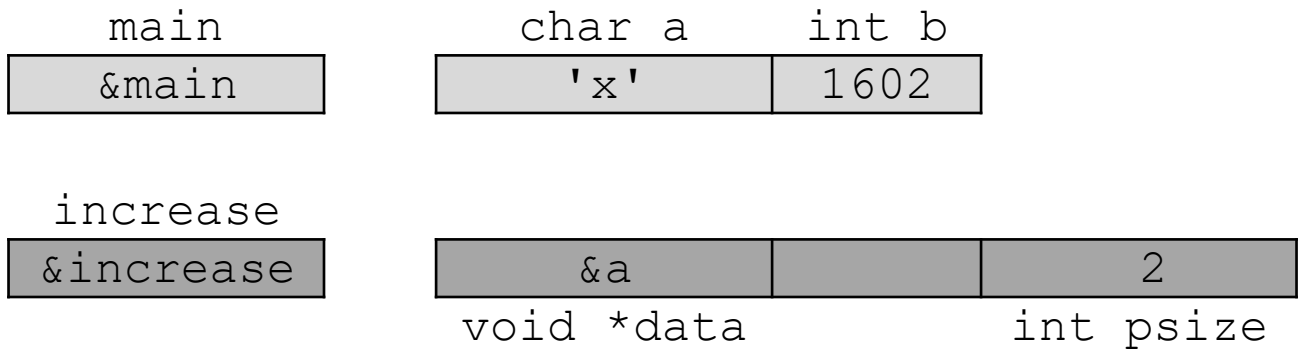
- We start with the `main` where two variables `char a` and `int b` is created with the values `'x'` and `1602` respectively (line 15-16).
- Line 17 calls the function `increase` with address of `a` and the size of `a` as parameters.
- `sizeof(a)=sizeof(char) = 2`, as `char` type is two bytes long.



void POINTERS

```
1  // increaser
2  void increase(void *data, int psize){
3    if ( psize == sizeof(char) ){
4       char *pchar;
5       pchar=(char*)data;
6       ++(*pchar);
7    }
8    else if (psize == sizeof(int)){
9       int *pint;
10      pint=(int*)data;
11      ++(*pint);
12   }
13 }
14 void main (void){
15   char a = 'x';
16   int b = 1602;
17   increase (&a,sizeof(a));
18   increase (&b,sizeof(b));
19   cout << a << ", " << b << endl;
20 }
```

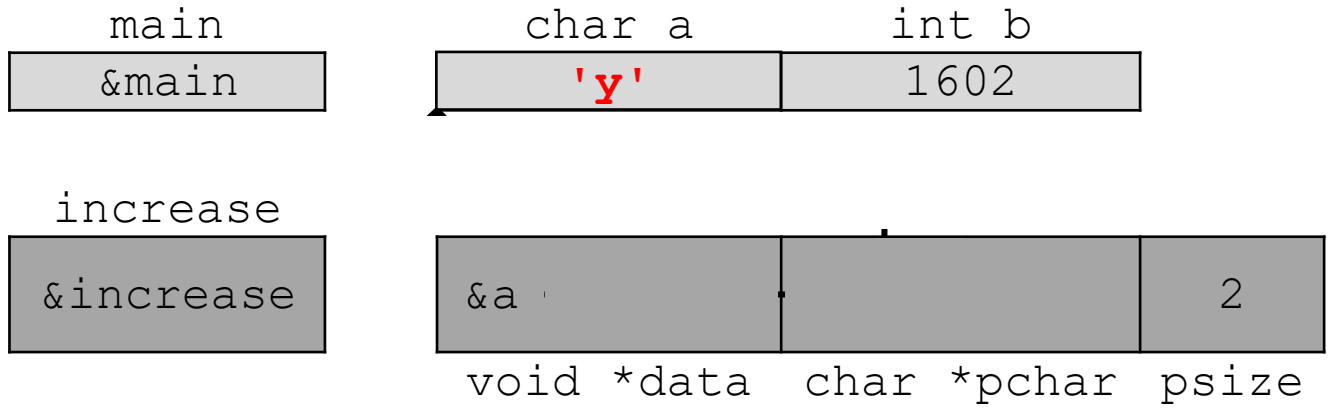
- Line 17 gives the control to the function `increase` and it creates two (parameter) variables `void *data` and `int psize` assigned with the address value of `a` and `sizeof(a)=2` of `main` respectively.
- Though `*data` contains the address of `a` in `main`, this address cannot be accessed using `*data` with type mismatch.



void POINTERS

```
1  // increaser
2  void increase(void *data, int psize){
3    if ( psize == sizeof(char) ){
4       char *pchar;
5       pchar=(char*)data;
6       ++(*pchar);
7    }
8    else if (psize == sizeof(int)){
9       int *pint;
10      pint=(int*)data;
11      ++(*pint);
12   }
13 }
14 void main (void){
15   char a = 'x';
16   int b = 1602;
17   increase (&a,sizeof(a));
18   increase (&b,sizeof(b));
19   cout << a << ", " << b << endl;
20 }
```

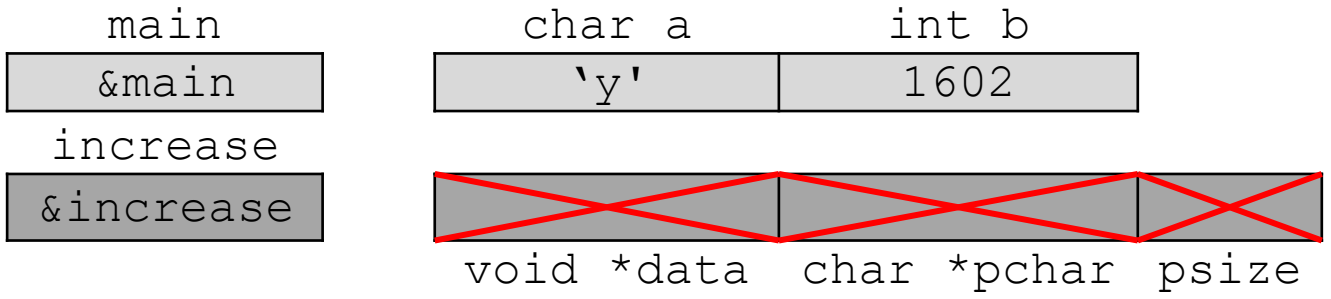
- With the `true` value of the conditional statement `psize==sizeof(char)` another new pointer variable `char *pchar` is created and assigned to the value, `*data` (line 3-5)
- As `*data` has no type, it must be ***type casted*** to `(char *)` before being assigned (line 5).
- With the statement `++(*pchar);` pointer variable `*pchar`, pointing to `a` of `main`, is increased by one. So, the value of a in `main` is changed to `'y'` (the `ASCII` value is increased by one) (line 6).



void POINTERS

```
1  // increaser
2  void increase(void *data, int psize){
3    if ( psize == sizeof(char) ){
4       char *pchar;
5       pchar=(char*)data;
6       ++(*pchar);
7    }
8    else if (psize == sizeof(int)){
9       int *pint;
10      pint=(int*)data;
11      ++(*pint);
12   }
13 }
14 void main (void){
15   char a = 'x';
16   int b = 1602;
17   increase (&a,sizeof(a));
18   increase (&b,sizeof(b));
19   cout << a << ", " << b << endl;
20 }
```

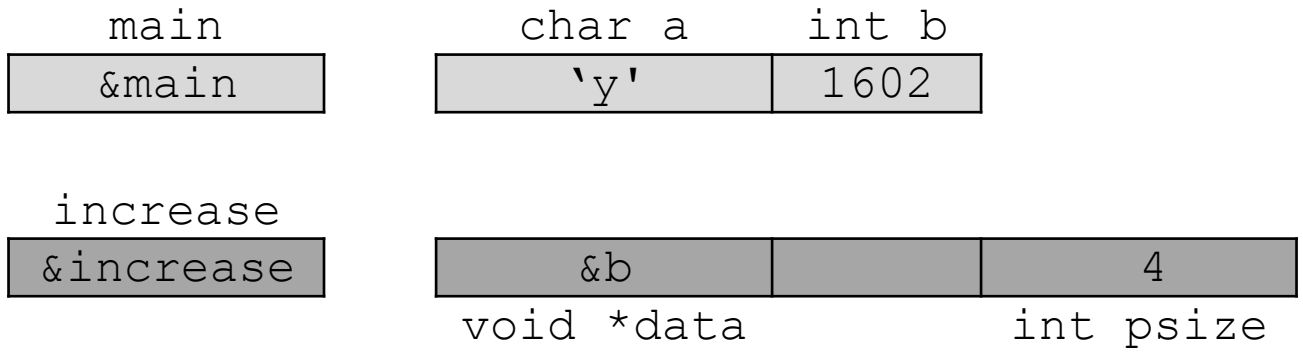
- Before exiting the function `increase`, the variables created by increase is destroyed.
- Then the control goes back to the function main (in line 17).
- So, we see the value `a` in `main` is changed to `'y'`.



void POINTERS

```
1  // increaser
2  void increase(void *data, int psize){
3    if ( psize == sizeof(char) ){
4       char *pchar;
5       pchar=(char*)data;
6       ++(*pchar);
7    }
8    else if (psize == sizeof(int)){
9       int *pint;
10      pint=(int*)data;
11      ++(*pint);
12   }
13 }
14 void main (void){
15   char a = 'x';
16   int b = 1602;
17   increase (&a,sizeof(a));
18   increase (&b,sizeof(b));
19   cout << a << ", " << b << endl;
20 }
```

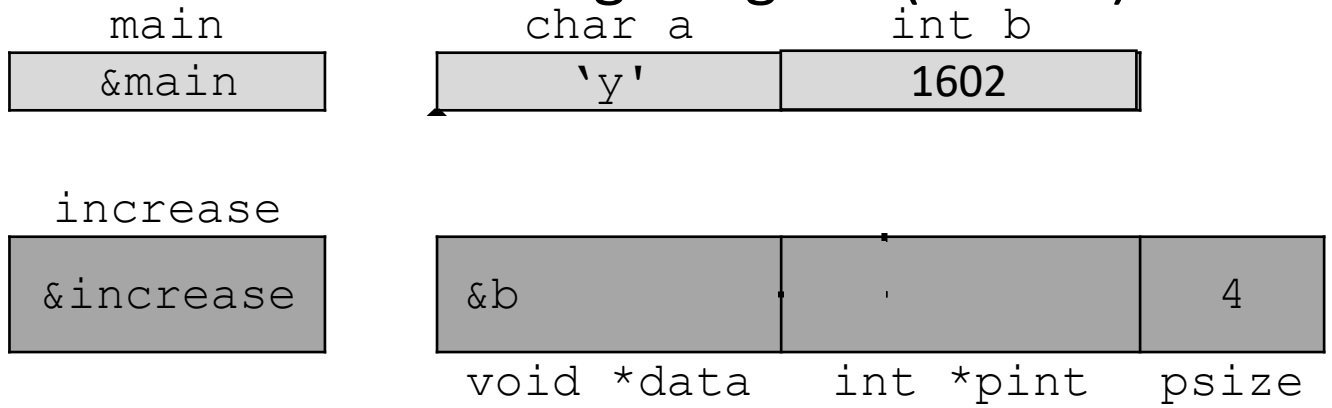
- Line 18 calls the function `increase` with address of `b` and the size of `b` as parameters.
- `sizeof(b)= sizeof(int)= 4`, as `int` type is four bytes long.
- Now the control goes to the function `increase` and it creates two (parameter) variables `void *data` and `int psize` assigned with the address value of `a` and `sizeof(b)=4` of `main` respectively.



void POINTERS

```
1  // increaser
2  void increase(void *data, int psize){
3    if ( psize == sizeof(char) ){
4       char *pchar;
5       pchar=(char*)data;
6       ++(*pchar);
7    }
8    else if (psize == sizeof(int)){
9       int *pint;
10      pint=(int*)data;
11      ++(*pint);
12   }
13 }
14 void main (void){
15   char a = 'x';
16   int b = 1602;
17   increase (&a,sizeof(a));
18   increase (&b,sizeof(b));
19   cout << a << ", " << b << endl;
20 }
```

- With the `true` value of the conditional statement `psize==sizeof(int)`, a new pointer variable `int *pint` is created and assigned to the value, `*data` (line 8-10) .
- Though `*data` contains the address of `b` in `main`, this address cannot be accessed using `*data` with type mismatch (line 10).
- As `*data` has no type, it must be type casted to `int *` before being assigned (line 10).

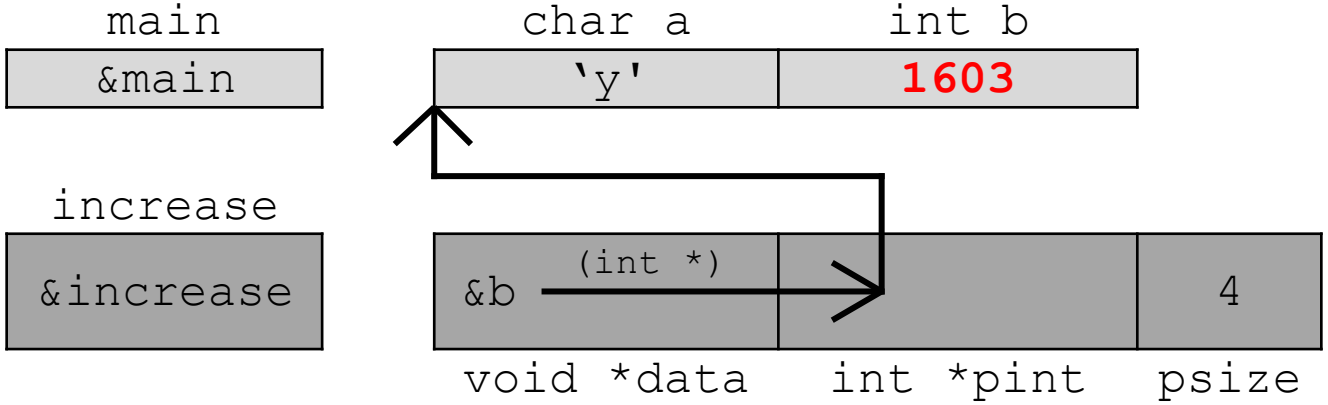


void POINTERS

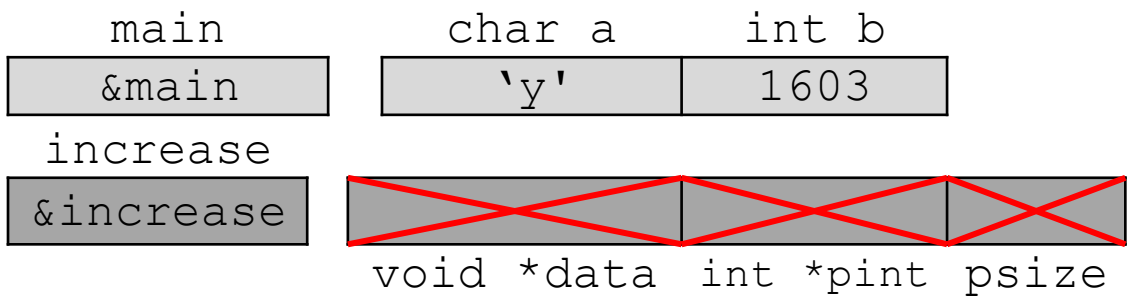
```
1  // increaser
2  void increase(void *data, int psize){
3    if ( psize == sizeof(char) ){
4       char *pchar;
5       pchar=(char*)data;
6       ++(*pchar);
7    }
8    else if (psize == sizeof(int)){
9       int *pint;
10      pint=(int*)data;
11      ++(*pint);
12   }
13 }
14 void main (void){
15   char a = 'x';
16   int b = 1602;
17   increase (&a,sizeof(a));
18   increase (&b,sizeof(b));
19   cout << a << ", " << b << endl;
20 }
```

```
Y, 1603
```

- With the statement `++(*pint);` pointer variable `*pint`, pointing to a of `main`, is increased by one. So, the value of a in `main` is changed to `1603` (line 11).



- Before exiting the function increase, the variables created by increase is destroyed. Then the control goes back to the function main (in line 17). So, we see the value `b` is changed to 1063.



NULL POINTER

- A `NULL` pointer is a regular pointer of any pointer type which has a special value that indicates that it is not pointing to any valid reference or memory address. This value is the result of type-casting the integer value zero to any pointer type.

```
1 int * p;
2 p = 0;  //can also write, p = NULL;
3 // p has a null pointer value
```

- Do not confuse `null` pointers with `void` pointers. A `null` pointer is a value that any pointer may take to represent that it is pointing to "nowhere", while a `void` pointer is a special type of pointer that can point to somewhere without a specific type.
- One refers to the value stored in the pointer itself and the other to the type of data it points to.

DYNAMIC MEMORY ALLOCATION

- The exact size of array is unknown until the compile time, i.e., time when a compiler compiles code written in a programming language into an executable form. The size of array declared initially can be sometimes insufficient and sometimes more than required. Also, what if we need a variable amount of memory that can only be determined during runtime?
- Dynamic memory allocation allows a program to obtain more memory space, while running or to release space when no space is required.
- C++ integrates the operators `new` and `delete` for *dynamic memory* allocation.

DYNAMIC MEMORY ALLOCATION - OPERATORS **new** AND **new []**

- In order to request dynamic memory we use the operator `new`. `new` is followed by a data type specifier. If a sequence of more than one memory block is required, the data type specifier is followed by the number of these memory blocks within brackets `[]`. It returns a pointer to the beginning of the new block of memory allocated. Syntax:

```
1 pointer = new vtype;
2 pointer = new vtype [number_of_elements];
```

- The first expression is used to allocate memory to contain one single element of type `vtype`. The second one is used to assign a block (an array) of elements of type `vtype`, where `number_of_elements` is an integer value representing the amount of these.

DYNAMIC MEMORY ALLOCATION - OPERATORS `delete` AND `delete[]`

- Since the necessity of dynamic memory is usually limited to specific moments within a program, once it is no longer needed it should be freed so that the memory becomes available again for other requests of dynamic memory. This is the purpose of the operator `delete`, whose format is:

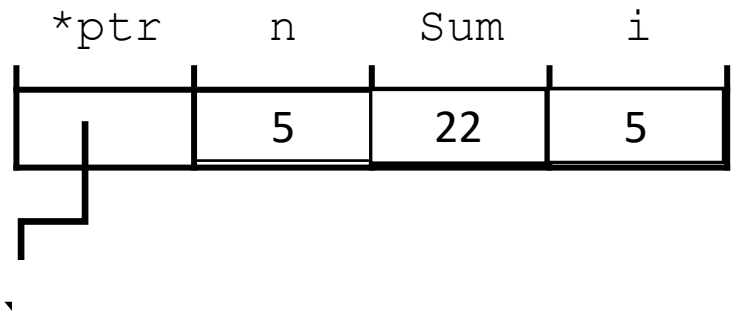
```
1 delete pointer;
2 delete [] pointer;
```

- The first expression should be used to delete memory allocated for a single element, and the second one for memory allocated for arrays of elements.
- The value passed as argument to delete must be either a pointer to a memory block previously allocated with `new`, or a `null` pointer (in the case of a `null` pointer, `delete` produces no effect).

DYNAMIC MEMORY ALLOCATION

```
1  // new, delete
2  void main(void){
3      int n, i,*ptr, sum=0;
4      cout << "# of elements: ";
5      cin >> n;          //input 5
6      ptr = new int[n];
7      if(ptr==NULL){  //ptr==0
8       cout << "Error! not allocated.";
9       exit(0);
10     }
11     cout << "Enter elements:\n";
12     for(i=0;i<n;++i)
13     {   //input 2 6 7 4 3
14         cin >> (ptr+i);   //ptr[i]
15         sum += *(ptr+i);  //ptr[i]
16     }
17     cout << "Sum = " << sum;
18     delete [] (ptr);
19     //memory de-allocated
20 }
```

```
# of elements: 5
Enter elements:
2
6
7
4
3
Sum = 22
```

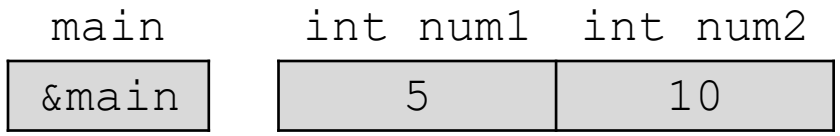


POINTER & FUNCTION

```
1  /* Swap two numbers using function. */
2  void swap(int *a, int *b);
3  void main(void){
4    int num1=5,num2=10;
5    swap(&num1,&num2);
6    /* address of num1, num2 is passed */
7    cout<<"Number1 = "<<num1<<"\n";
8    cout<<"Number2 = "<<num2;
9  }
10 void swap(int *a, int *b){
11 //a,b points to &num1,&num2 respectively
12   int t;
13   t = *a;
14   *a = *b;
15   *b = t;
16 }
```

```
Number1 = 10
Number2 = 5
```

- Passing arguments to functions by value, there is no direct way for the called function to alter a variable in the calling function.
- Pointer arguments enable a function to access and change objects in the function that called it. Let's consider the example on the left.
- The program starts with function main getting the control and creating two variables `num1` and `num2` with values `5` and `10` respectively (line 4).

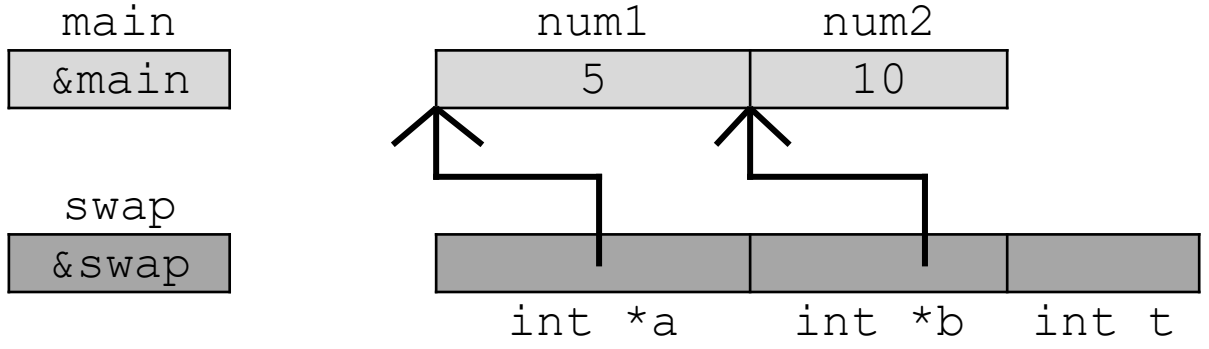


POINTER & FUNCTION

```
1  /* Swap two numbers using function. */
2  void swap(int *a, int *b);
3  void main(void){
4    int num1=5,num2=10;
5    swap(&num1,&num2);
6    /* address of num1, num2 is passed */
7    cout<<"Number1 = "<<num1<<"\n";
8    cout<<"Number2 = "<<num2;
9  }
10 void swap(int *a, int *b){
11 //a,b points to &num1,&num2 respectively
12   int t;
13   t = *a;
14   *a = *b;
15   *b = t;
16 }
```

```
Number1 = 10
Number2 = 5
```

- Function `main` calls the function `swap` with two parameter values `&num1` and `&num2` (line 5).
- So the address of `num1` and `num2` is send through the parameter of `swap`.
- Now the control goes to the function `swap` and it creates two pointer (parameter) variables `*a` and `*b` assigned with the address values of `num1` and `num2` of `main` respectively.
- Another variable `t` is created (line 12).

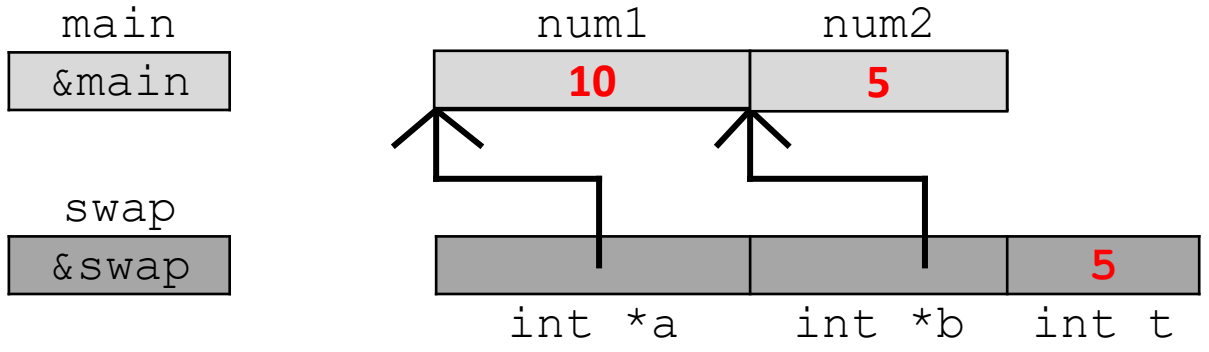


POINTER & FUNCTION

```
1  /* Swap two numbers using function. */
2  void swap(int *a, int *b);
3  void main(void){
4    int num1=5,num2=10;
5    swap(&num1,&num2);
6    /* address of num1, num2 is passed */
7    cout<<"Number1 = "<<num1<<"\n";
8    cout<<"Number2 = "<<num2;
9  }
10 void swap(int *a, int *b){
11 //a,b points to &num1,&num2 respectively
12   int t;
13   t = *a;
14   *a = *b;
15   *b = t;
16 }
```

```
Number1 = 10
Number2 = 5
```

- With the statement `t=*a;` variable `t` is assigned to the value, `num1` of `main`, pointed to by pointer `*a` (line 13).
- With the statement `*a = *b;` pointer variable `*a`, pointing to `num1` of `main`, is assigned to the value, `num2` in `main`, pointed to by pointer `*b` (line 14).
- With the statement `*b = t;` pointer variable `*b`, pointing to `num2` of `main`, is assigned to the value of `t` (line 15).

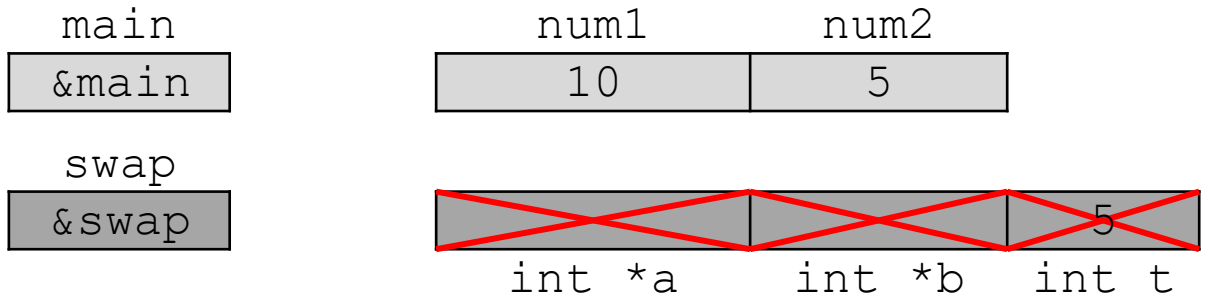


POINTER & FUNCTION

```
1  /* Swap two numbers using function. */
2  void swap(int *a, int *b);
3  void main(void){
4    int num1=5,num2=10;
5    swap(&num1,&num2);
6    /* address of num1, num2 is passed */
7    cout<<"Number1 = "<<num1<<"\n";
8    cout<<"Number2 = "<<num2;
9  }
10 void swap(int *a, int *b){
11 //a,b points to &num1,&num2 respectively
12   int t;
13   t = *a;
14   *a = *b;
15   *b = t;
16 }
```

```
Number1 = 10
Number2 = 5
```

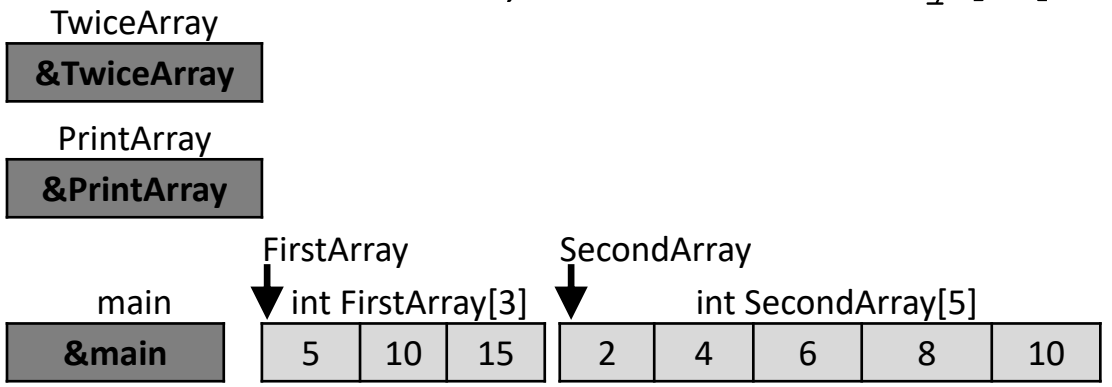
- Any changes we make to `*a` and `*b` in function `swap`, will change the values of `num1` and `num2` respectively in function `main` as `a` is pointing to `num1` and `b` is pointing to `num2`.
- Before exiting the function `swap`, the variables created by `swap` is destroyed. Then the control goes back to the function `main` (in line 5). But nothing changes for the values of the variables `num1` and `num2` of `main` due to the destruction of the variables `*a` and `*b`. Because, destruction only destroys the space provided for `*a` and `*b`. It does not destroy the space it was pointing to. Whatever changes were made in swap by `*a` and `*b` remains.



POINTER, ARRAY & FUNCTION

```
1  // arrays as parameters
2  void TwiceArray (int arg[], int length) {
3    for (int n=0; n<length; n++) arg[n] *= 2;
4  }
5  void PrintArray(const int arg[], int length){
6    for (int n=0; n<length; n++)
7      printf("%d ", arg[n]);
8    printf("\n");
9  }
10 void main (void){
11   int FirstArray[3] = {5, 10, 15};
12   int SecondArray[] = {2, 4, 6, 8, 10};
13   TwiceArray (FirstArray,3);
14   PrintArray (FirstArray,3);
15   PrintArray (SecondArray,5);
16 }
```

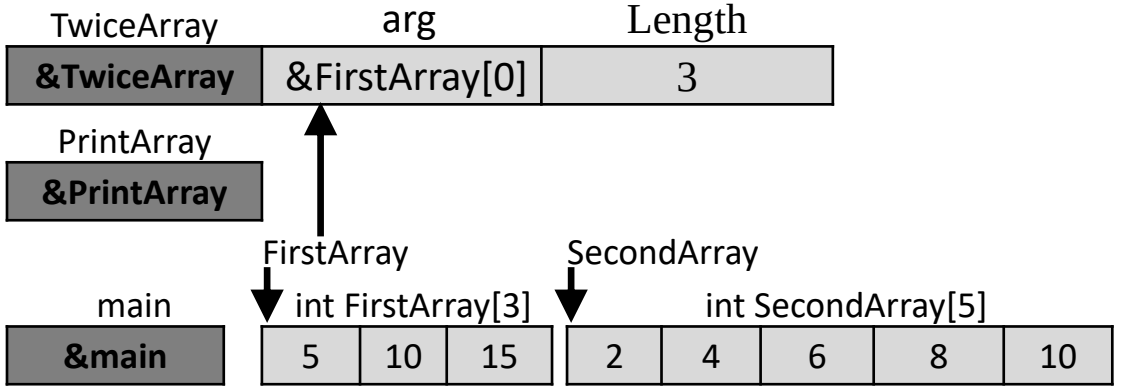
- Then we start with the function `main` where two arrays, `FirstArray` with 3 elements and `SecondArray` with 5 elements, are declared, created, and initialized (line 11-12).
- Both these identifiers will hold the starting address/location of their elements.
- `FirstArray` holds the location of the first element, `&FirstArray[0]` and
- `SecondArray` holds the location of the first element, `&SecondArray[0]`



POINTER, ARRAY & FUNCTION

```
1 // arrays as parameters
2 void TwiceArray (int arg[], int length) {
3   for (int n=0; n<length; n++) arg[n] *= 2;
4 }
5 void PrintArray(const int arg[], int length){
6   for (int n=0; n<length; n++)
7     printf("%d ", arg[n]);
8   printf("\n");
9 }
10 void main (void){
11   int FirstArray[3] = {5, 10, 15};
12   int SecondArray[] = {2, 4, 6, 8, 10};
13   TwiceArray (FirstArray,3);
14   PrintArray (FirstArray,3);
15   PrintArray (SecondArray,5);
16 }
```

- Then, the `TwiceArray` is called with the parameters `FirstArray`, the starting location of the array itself or the location of the first element is passed to the parameter `int arg[]` and value `3` to the parameter `length` (line 13).
- The identifier `arg` holds the starting address of the `FirstArray` of the function main. As `arg` itself is an array, `arg` behaves like an array.
- The control is transferred from function main to function `TwiceArray` (line 2).

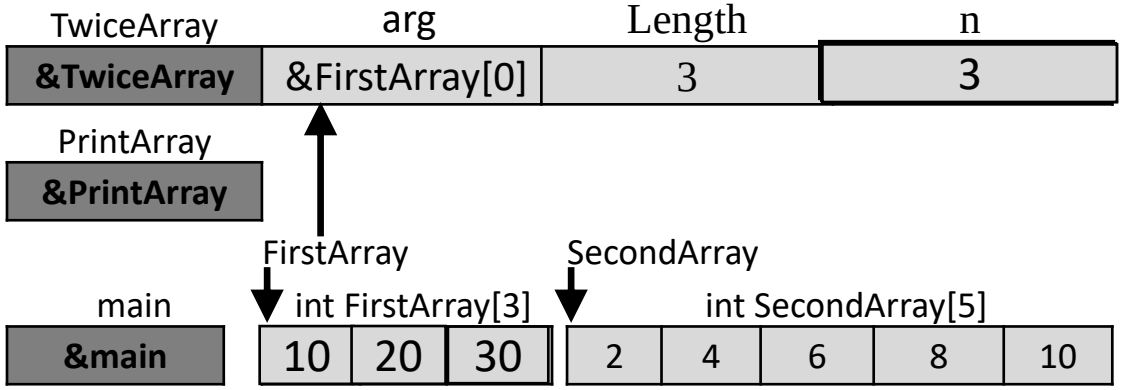


POINTER, ARRAY & FUNCTION

```
1  // arrays as parameters
2  void TwiceArray (int arg[], int length) {
3    for (int n=0; n<length; n++) arg[n] *= 2;
4  }
5  void PrintArray(const int arg[], int length){
6    for (int n=0; n<length; n++)
7      printf("%d ", arg[n]);
8    printf("\n");
9  }
10 void main (void){
11   int FirstArray[3] = {5, 10, 15};
12   int SecondArray[] = {2, 4, 6, 8, 10};
13   TwiceArray (FirstArray,3);
14   PrintArray (FirstArray,3);
15   PrintArray (SecondArray,5);
16 }
```

- Inside function `TwiceArray` another variable `n` is declared. Using `n` in `for` loop `3` elements of `arg` are made twice of their original (line 3).
- As `arg` represents the array `FirstArray` on `main`, the `3` elements of the `FirstArray` are actually made twice.
- Hypothetically,

  `FirstArray[n]←arg[n]*=2.`



POINTER, ARRAY & FUNCTION

```
1  // arrays as parameters
2  void TwiceArray (int arg[], int length) {
3    for (int n=0; n<length; n++) arg[n] *= 2;
4  }
5  void PrintArray(const int arg[], int length){
6    for (int n=0; n<length; n++)
7      printf("%d ", arg[n]);
8    printf("\n");
9  }
10 void main (void){
11   int FirstArray[3] = {5, 10, 15};
12   int SecondArray[] = {2, 4, 6, 8, 10};
13   TwiceArray (FirstArray,3);
14   PrintArray (FirstArray,3);
15   PrintArray (SecondArray,5);
16 }
```

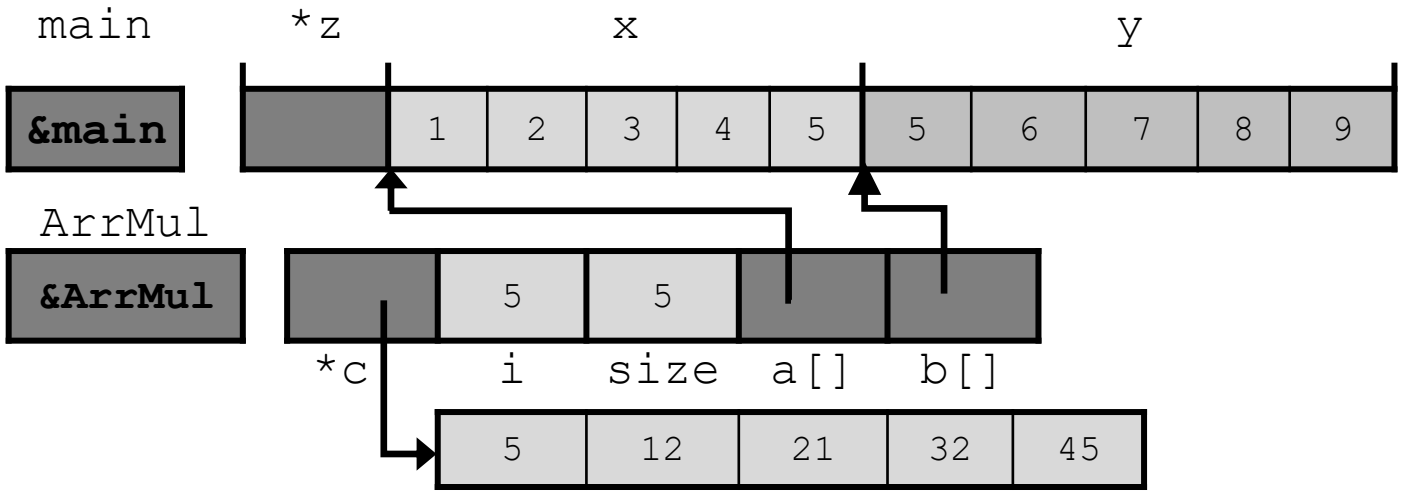
- Before exiting from the function `TwiceArray`, all the variables (`arg`, `length`, `n`) are destroyed and the control is returned to the `main` (line 13). Values of `FirstArray` changed in `TwiceArray` remains.
- Next the function `PrintArray` is called to print the elements of the `FirstArray` and `SecondArray` consecutively. Here, it works as same in terms of parameter passing. Except the parameter `arg` in `PrintArray` which is declared as `constant` variable.
- As we do not need to change any elements of the array inside `PrintArray`, the parameter `arg` is declared as `constant` variable.
- This is how any function (`main` in this example) can protect its data array (`FirstArray`) from being changed by another function (`PrintArray`).

```
1  //Array Multiplication
2  int *ArrMul(int a[], int b[], int size){
3    int i,*c = new int[size]; //c[5]
4    for(i=0; i<size; i++) c[i] = a[i] *
5  b[i];
6    return c;
7  }
8  void PrintArr(int *a, int size){
9    for(int i=0; i<size; i++)
10 cout<<a[i]<<"\t";
11   cout << "\n ";
12 }
13 void main(void){
14   int
15 *z,x[5]={1,2,3,4,5},y[5]={5,6,7,8,9};
16   z = ArrMul(x, y, 5);
17   cout <<"First array:\n";
18   PrintArr( x, 5 );
19   cout <<"second array:\n";
20   PrintArr( y, 5 );
21   cout <<"result array:\n";
     PrintArr( z, 5 );
     delete [] (z); //memory deallocated
   }
```

```
First array:
1    2    3    4    5
second array:
5    6    7    8    9
result array:
5    12   21   32   45
```

POINTER, ARRAY & FUNCTION

- Two array `x` and `y` with five elements are multiplied index wise using the function `ArrMul`.
- Function `ArrMul` (line 2-6) dynamically allocates memory for the resultant array `c` of the multiplication.

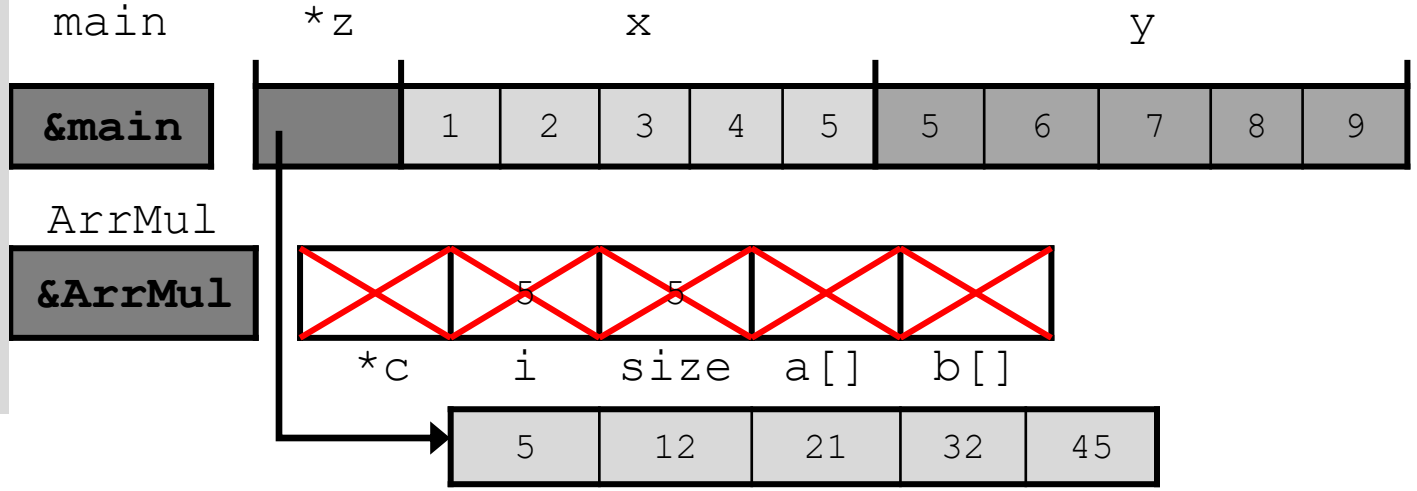


POINTER, ARRAY & FUNCTION

```
1  //Array Multiplication
2  int *ArrMul(int a[], int b[], int size){
3    int i,*c = new int[size]; //c[5]
4    for(i=0; i<size; i++) c[i] = a[i] * b[i];
5    return c;
6  }
7  void PrintArr(int *a, int size){
8    for(int i=0; i<size; i++) cout<<a[i]<<"\t";
9    cout << "\n ";
10 }
11 void main(void){
12   int *z,x[5]={1,2,3,4,5},y[5]={5,6,7,8,9};
13   z = ArrMul(x, y, 5);
14   cout <<"First array:\n";
15   PrintArr( x, 5 );
16   cout <<"second array:\n";
17   PrintArr( y, 5 );
18   cout <<"result array:\n";
19   PrintArr( z, 5 );
20   delete [] (z); //memory deallocated
21 }
```

```
First array:
1    2    3    4    5
second array:
5    6    7    8    9
result array:
5    12   21   32   45
```

- Before exit from function `ArrMul` the address stored in `*c` returned and all the variables created in `ArrMul` is destroyed.
- The control is transferred to `main` and `z` is assigned to the address value returned by `c` of `ArrMul` (line 13).
- Then the arrays represented by `x`, `y`, and `z` is printed using function `PrintArr` (line 14-19).

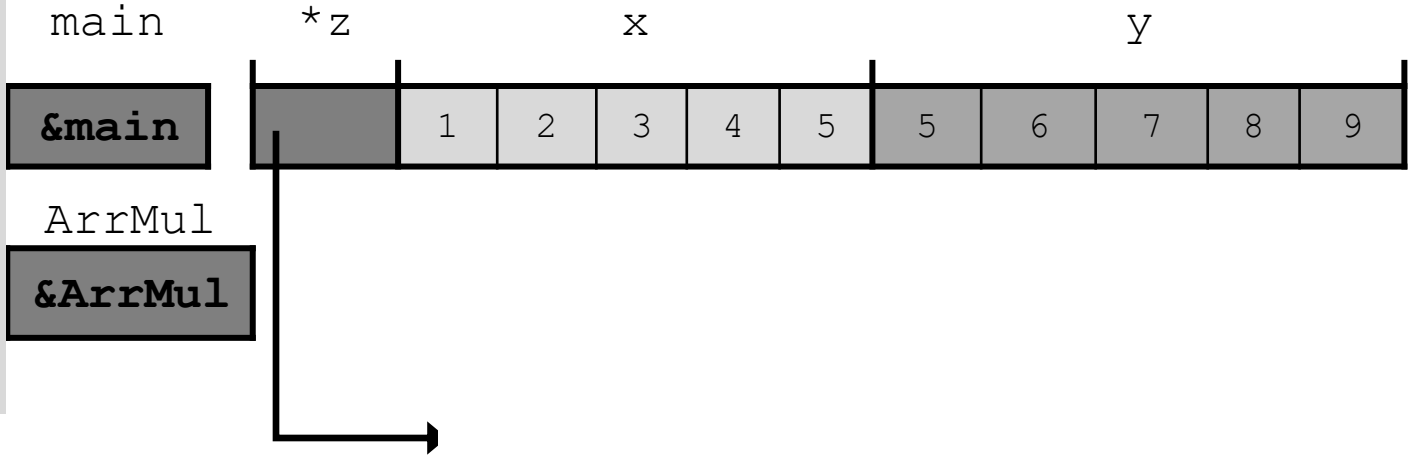


POINTER, ARRAY & FUNCTION

```
1  //Array Multiplication
2  int *ArrMul(int a[], int b[], int size){
3    int i,*c = new int[size]; //c[5]
4    for(i=0; i<size; i++) c[i] = a[i] * b[i];
5    return c;
6  }
7  void PrintArr(int *a, int size){
8    for(int i=0; i<size; i++) cout<<a[i]<<"\t";
9    cout << "\n ";
10 }
11 void main(void){
12   int *z,x[5]={1,2,3,4,5},y[5]={5,6,7,8,9};
13   z = ArrMul(x, y, 5);
14   cout <<"First array:\n";
15   PrintArr( x, 5 );
16   cout <<"second array:\n";
17   PrintArr( y, 5 );
18   cout <<"result array:\n";
19   PrintArr( z, 5 );
20   delete [] (z); //memory deallocated
21 }
```

```
First array:
1    2    3    4    5
second array:
5    6    7    8    9
result array:
5    12   21   32   45
```

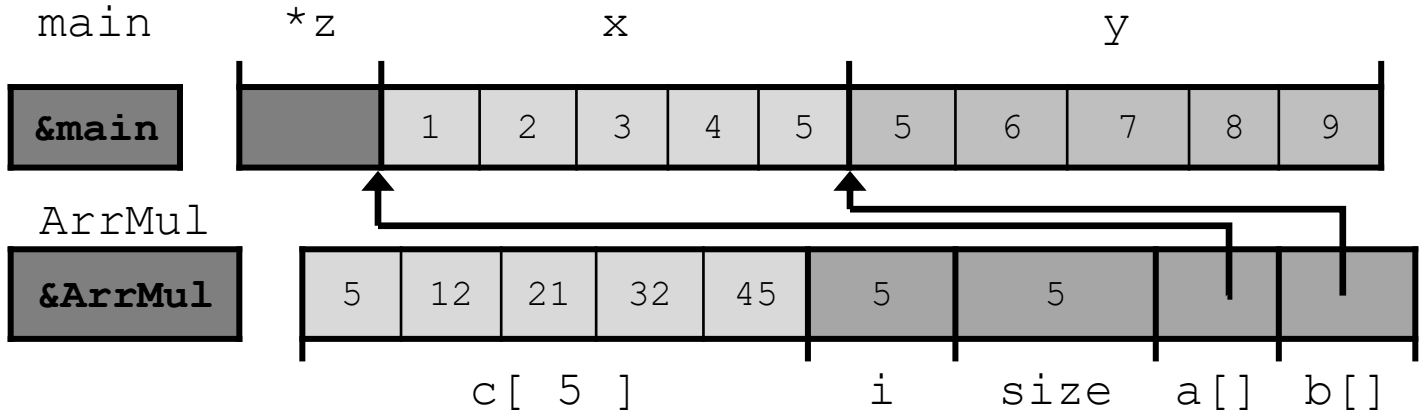
- Line 20 de-allocates the memory allocated to `*c` in function `ArrMul` (line 3) and later returned to `*z` in function `main` (line 13).
- A dynamically allocated memory must be de-allocated.



POINTER, ARRAY & FUNCTION

```
1  //Array Multiplication
2  int *ArrMul(int a[], int b[], int size){
3    int i, c[5];
4    for(i=0; i<size; i++) c[i] = a[i] * b[i];
5    return c;
6  }
7  void PrintArr(int *a, int size){
8    for(int i=0; i<size; i++) cout<<a[i]<<"\t";
9    cout << "\n ";
10 }
11 void main(void){
12   int *z,x[5]={1,2,3,4,5},y[5]={5,6,7,8,9};
13   z = ArrMul(x, y, 5);
14   cout <<"First array:\n";
15   PrintArr( x, 5 );
16   cout <<"second array:\n";
17   PrintArr( y, 5 );
18   cout <<"result array:\n";
19   PrintArr( z, 5 );
20   delete [] (z); //memory deallocated
21 }
```

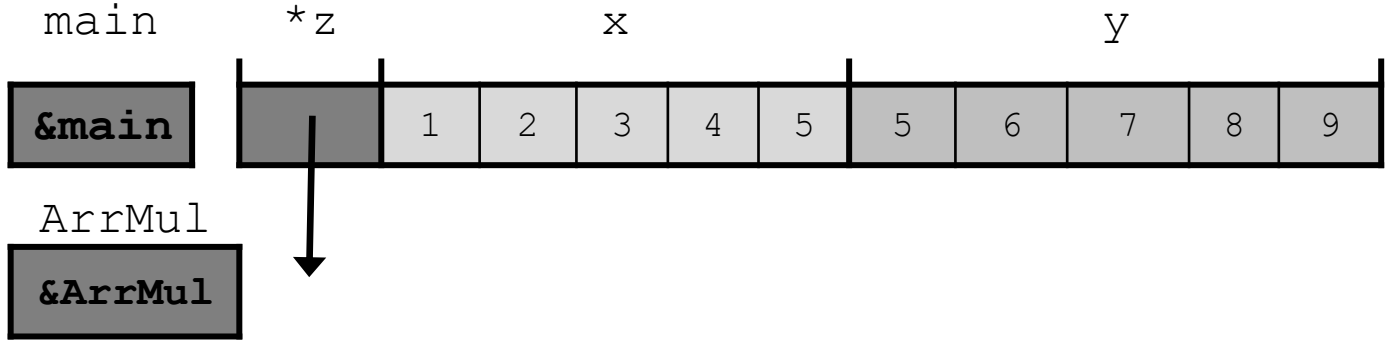
- Consider the case, if dynamic array `*c` in `ArrMul` was declared as an array `c[5]`.
- That is, instead of `int *c=new int[size]` in line 3, if it was `int c[5]`, the following would happen.
- The address represented by `c` is returned and all the variables created in `ArrMul` is destroyed and control is transferred to `main` at exit from `ArrMul`.



POINTER, ARRAY & FUNCTION

```
1  //Array Multiplication
2  int *ArrMul(int a[], int b[], int size){
3    int i, c[5];
4    for(i=0; i<size; i++) c[i] = a[i] * b[i];
5    return c;
6  }
7  void PrintArr(int *a, int size){
8    for(int i=0; i<size; i++) cout<<a[i]<<"\t";
9    cout << "\n ";
10 }
11 void main(void){
12   int *z,x[5]={1,2,3,4,5},y[5]={5,6,7,8,9};
13   z = ArrMul(x, y, 5);
14   cout <<"First array:\n";
15   PrintArr( x, 5 );
16   cout <<"second array:\n";
17   PrintArr( y, 5 );
18   cout <<"result array:\n";
19   PrintArr( z, 5 );
20   delete [] (z); //memory deallocated
21 }
```

- The address represented by `c` is returned and all the variables created in `ArrMul` is destroyed and control is transferred to `main` at exit from `ArrMul`.
- `*z` is assigned to the address value returned by `c` (line 13).
- `PrintArr` finds an error when trying to print the array `z` (line 19), as the address represented by `z` has already been destroyed at the exit of the function `ArrMul`.
- So, while returning a pointer to any variable or memory area, must make sure that the returned memory area is active or not destroyed after return.



POINTERS AND INITIALIZATION

- Consider the statement `int *p;` which declares a pointer `p`, and like any other variable, this space will contain garbage (random numbers), because no statement like `p = &someint;` or `p = new int;` has yet been encountered which would give it a value.
- Writing a statement `int *p=2000;` is syntactically correct as `p` will point to the 2000th byte of the memory. But it might fail as byte 2000 might be being used by some other program or may be being used by some other data type variable of the same program. So such initialization or assignment must be avoided unless the address provided is guaranteed to be safe.
- There is an important difference between these definitions:

```
char amsg[] = "now is the time"; /* an array */
char *pmsg = "now is the time"; /* a pointer */
```

  - `amsg` is an array, just big enough to hold the sequence of characters and `'\0'` that initializes it. Individual characters within the array may be changed but `amsg` will always refer to the same storage and size after declaration and initialization.
  - On the other hand, `pmsg` is a pointer, initialized to point to a string constant; the pointer may subsequently be modified to point elsewhere, but the result is undefined if you try to modify the string contents.